

# Sharp-polygon overlap: area, gradient, and implementation

A self-contained reference tying the straight-edged (no-rounding) overlap *area* and *gradient* formulas (Cam’s `definitions.tex`, §3–4) to their implementation in `polygonOverlap.py`, with the validation status and the non-smoothness we hit. Everything here is the sharp model: features are the  $n$  edges of each polygon, the only crossings are edge–edge, and a boundary point’s velocity is a plain vertex velocity (no corner Jacobians). [Blue marks my own remarks / corrections to the handwritten notes.](#)

## 1 Data structures

A packing stores flat vertices  $\vec{v}_k$ ,  $k \in [0, N_v)$ , with CSR `startIndices` and `next/prev` (so edge  $k$  runs  $\vec{v}_k \rightarrow \vec{v}_{z(k)}$ ,  $z(k) = \text{next}[k]$ ). All polygons are CCW. Two quantities are attached to the packing by “update” functions:

`packing.intersections` an  $(M, 6)$  array; row = one edge–edge crossing, columns

$$(\rho_i, \rho_j, j, s_j, i, s_i),$$

where  $\rho_i, \rho_j$  are the two polygon indices,  $i, j$  are *global* edge (start-vertex) indices, and  $s_i, s_j \in (0, 1)$  the fraction along each edge. The record is *oriented*: edge  $i$  is the one whose forward (+ $s$ ) direction *enters* the other polygon — it continues the CCW overlap boundary — and  $j$  is the other edge. [We store  \$\rho\$  \(topology, constant\) even though  \$\rho\_i = \text{shapeId}\[i\]\$ : the row \*is\* the §5 sort key, so sorting/pairing is a lexicographic sort with no gather, and keeping  \$\(i, s\_i\)\$  separate \(not a fused  \$c\_i = i + s\_i\$ \) preserves full float precision on  \$s\$  regardless of  \$N\_v\$ .](#)

`packing.followerIndices` an  $(M, )$  int array; `followerIndices[ $\alpha$ ]` is the index of the intersection that *follows*  $\alpha$  along the CCW overlap boundary.

## 2 Intersections

**Two edges (polygonPairIntersections).** Edges  $i = (\vec{v}_i, \vec{e}_i)$  and  $j = (\vec{v}_j, \vec{e}_j)$  cross where  $\vec{v}_i + s\vec{e}_i = \vec{v}_j + t\vec{e}_j$ . Solving,

$$s = \frac{(\vec{v}_j - \vec{v}_i) \times \vec{e}_j}{\vec{e}_i \times \vec{e}_j}, \quad t = \frac{(\vec{v}_i - \vec{v}_j) \times \vec{e}_i}{\vec{e}_j \times \vec{e}_i}, \quad \vec{a} \times \vec{b} = a_x b_y - a_y b_x, \quad (1)$$

a genuine crossing needing  $s, t \in (0, 1)$  (parallel edges,  $\vec{e}_i \times \vec{e}_j = 0$ , skipped). [This is the \*corrected\* sign of `definitions.tex` eq \(2.2\)/\(2.3\), whose numerators are flipped \( \$\vec{v}\_i - \vec{v}\_j\$  where it should be  \$\vec{v}\_j - \vec{v}\_i\$  for  \$s\$ \). The code has the right sign, which is why the MC area check passes.](#) In code, with A the first loop and B the second, `sA=cross(w,dB)/cross(dA,dB)`, `w=rB[b]-rA[a]`.

**Assembly (updateIntersections).** All-to-all over polygon pairs  $(A, B)$ ,  $A < B$  (the neighbor-list version is a later drop-in).  $B$  is brought into  $A$ 's minimum image (`minImageShift`), edges are tested pairwise, and each crossing is written as a global oriented record. Orientation is a pure sign test: since  $B$  is CCW its interior is to the left of edge  $b$ , so *edge  $A$  enters  $B$  iff  $\vec{e}_B \times \vec{e}_A > 0$*  — no point-in-polygon needed. If  $A$  enters,  $(\rho_i, \rho_j, i, j) = (A, B, \text{edge}_A, \text{edge}_B)$ ; else the roles swap.

**Pairing (updateFollowers).** At intersection  $\alpha$  we leave along edge  $i$  on polygon  $\rho_i$  at continuous coordinate  $c = i + s_i$ . Walking  $\rho_i$ 's boundary forward, the next crossing of the *same* pair is the follower: the intersection  $\beta$  with the polygons swapped ( $\rho_i[\beta] = \rho_j[\alpha]$ ,  $\rho_j[\beta] = \rho_i[\alpha]$ ) whose arrival coordinate on  $\rho_i$  (its  $j$  field,  $l + s_l$ ) has the smallest forward distance mod  $n_{\rho_i}$  from  $c$ . The followers form a permutation whose cycles are the overlap-region boundary loops.  *$O(M^2)$  reference scan; the notes' sort + binary search is the fast version (TODO).*

### 3 Overlap area

The area of  $A \cap B$  is the shoelace boundary integral, discretized into chord triangles from a fixed reference  $\vec{R}$ :

$$|A \cap B| = \frac{1}{2} \varepsilon_{\alpha\beta} \oint_{\partial(A \cap B)} X^\alpha dX^\beta, \quad h(\vec{P}, \vec{Q}) = \frac{1}{2} (\vec{P} - \vec{R}) \times (\vec{Q} - \vec{R}). \quad (2)$$

$\vec{R}$  is the lowest-indexed vertex of the lower-indexed polygon of the pair (the  $R$ -dependence cancels over a closed loop, so the total is  $R$ -independent). Walking from a crossing  $\alpha$  (point  $\vec{q}_i$  on edge  $i$ ) to its follower  $\beta$  (point  $\vec{q}_l$  on edge  $l$ , both on  $\rho_i$ ), the run contributes

$$\underbrace{h(\vec{q}_i, \vec{q}_l)}_{\text{same edge}} \quad \text{or} \quad \underbrace{h(\vec{q}_i, \vec{v}_{z(i)}) + h(\vec{v}_l, \vec{q}_l)}_{\text{end caps } (U_{\text{ex}})} + \underbrace{\sum_{m \text{ strictly between } i, l} h(\vec{v}_m, \vec{v}_{z(m)})}_{\text{full interior edges } (U_{\text{int}})}. \quad (3)$$

Summed over all runs this is the total overlap area, split for parallelism into

$$U_{\text{ex}} = \sum_{\alpha} [\text{end caps} / \text{single chord of run } \alpha] \quad (\text{one thread per intersection}), \quad (4)$$

$$U_{\text{int}} = \sum_{m=0}^{N_v-1} \sum_{\text{runs } p \ni m} h(\vec{v}_m, \vec{v}_{z(m)}) \quad (\text{one thread per edge, } \delta_{pm}), \quad (5)$$

$$U = U_{\text{ex}} + U_{\text{int}}. \quad (6)$$

**Implementation (updateOverlapArea).** `AEx` loops over intersections: with  $\vec{q}_i = \vec{v}_i + \text{shift} + s_i \vec{e}_i$  and  $\vec{q}_l$  likewise, it adds the single chord (if  $i = l$  and  $s_i \leq s_l$ ) or the two end caps. `AInt` loops over every edge  $m$ : for each run  $\alpha$  on  $m$ 's polygon it tests membership

$$0 < (m - i) \bmod n_{\rho} < \text{span}, \quad \text{span} = \begin{cases} (l - i) \bmod n_{\rho} & i \neq l \\ 0 \text{ or } n_{\rho} & i = l, \ s_i \leq s_l \end{cases}$$

and if inside adds  $h(\vec{v}_m, \vec{v}_{z(m)})$  with that pair's  $\vec{R}$  and min-image shift. The  $U_{\text{int}}$  loop is genuinely per-edge (no boundary walk) — an earlier version walked the interior edges serially and was replaced.

**Validation.** MC (image-aware  $\int \binom{k}{2} dx$ ,  $k$  = polygons covering a point): analytic 0.42377 vs MC 0.42397 (diff  $2 \times 10^{-4}$ , sampling noise).

## 4 Overlap gradient (the force)

Differentiating the boundary integral, the intersection-point motion cancels around the closed loop (Hadamard / the §4 trick), leaving a boundary term:

$$\delta A_{\cap} = \oint_{\partial(A \cap B)} \delta \vec{X} \times d\vec{X}. \quad (7)$$

On a segment of edge  $m$  from  $\lambda_0$  to  $\lambda_f$ ,  $\vec{X}(\lambda) = \vec{v}_m + \lambda \vec{e}_m$  so  $\delta \vec{X} = (1 - \lambda) \delta \vec{v}_m + \lambda \delta \vec{v}_{z(m)}$ , and integrating,

$$\boxed{\frac{\partial A_{\cap}}{\partial v_k^{\alpha}} = \sum_{\text{segments } m} [(1 - \bar{s}_m) \delta_{mk} + \bar{s}_m \delta_{z(m)k}] \Delta s_m \varepsilon_{\alpha\beta} e_m^{\beta}}, \quad \bar{s}_m = \frac{\lambda_0 + \lambda_f}{2}, \quad \Delta s_m = \lambda_f - \lambda_0.} \quad (8)$$

Each boundary segment deposits  $(1 - \bar{s}) \Delta s (\varepsilon \vec{e}_m)$  on its start vertex  $m$  and  $\bar{s} \Delta s (\varepsilon \vec{e}_m)$  on its end vertex  $z(m)$ , with  $(\varepsilon \vec{e}_m) = (e_m^y, -e_m^x)$ . The segments are exactly those of §3: end caps ( $\lambda$ :  $s_i \rightarrow 1$  and  $0 \rightarrow s_l$ ) and full interior edges ( $\lambda$ :  $0 \rightarrow 1$ , so  $\bar{s} = \frac{1}{2}$ ,  $\Delta s = 1$ ). **Crucially eq (4.3) needs *no* reference  $\vec{R}$  and *no* min-image shift — it depends only on edge vectors and the  $s$  parameters. The physical force is  $-\partial A_{\cap} / \partial \vec{v}$ .**

**Implementation** (`updateOverlapGradient`). A `deposit(edge, 10, 1f)` helper does exactly the boxed rule; it is called once per end cap (per intersection, mirror of  $U_{\text{ex}}$ ) and once per full interior edge (per edge, mirror of  $U_{\text{int}}$ ). The result is stored in `packing.overlapGradient` =  $\partial A_{\cap} / \partial \vec{v}$ .

**Validation.** Central-difference vs  $d(\text{overlapArea})/d\vec{v}$  at a generic config:  $\max |\text{err}| = 5.3 \times 10^{-10}$ , median  $9.8 \times 10^{-11}$ , net gradient  $4 \times 10^{-16}$  (translation invariance).

## 5 Smoothness — the wall

The overlap area is continuous ( $C^0$ ) in the vertices, but its *gradient is discontinuous* at a vertex–edge crossing event (a vertex passing through another polygon’s edge): a wedge of overlap appears/vanishes at the corner, so  $dA/dv$  jumps. Measured directly at a stalled config, vertex 20: one-sided slopes were +0.075 (forward) vs −0.012 (backward) in  $x$ , +0.0007 vs −0.043 in  $y$  — a genuine kink, not a bug (only 4/36 vertices, all on the topology boundary; the other 32 FD to  $10^{-9}$ ). Consequences:

- A line-search minimizer (CG, L-BFGS) walks onto such a kink and *hard-stalls*: no descent direction exists there. Our CG reaches machine precision on any smooth energy but floors at  $\max |F| \approx 3.5 \times 10^{-2}$  here; FIRE creeps lower ( $\sim 8 \times 10^{-3}$ ) only because it is damped dynamics, not a line search.
- So the sharp model *did not* remove the non-smoothness — it traded the rounded model’s divergent contact stiffness ( $d^{-1/2}$  at arc contacts) for vertex–edge force *jumps*. Neither exact-overlap energy is smooth enough for a descent method to reach machine precision.
- A genuinely  $C^{\infty}$  energy needs the mollified / diffuse-interface overlap (`smoothOverlap.tex`); that is the route if machine-precision minimization is the goal.

**Other known gaps.** Exactly-collinear shared edges or a crossing landing on a vertex are skipped by the strict  $0 < s < 1$  test (measure-zero); pure containment (one polygon inside another, no crossing) returns 0 (cannot occur in a dense equal-size packing). See `TODO.md`.